CURRENT IMPLEMENTATION + PI ROADMAP

StockAgent

Technical design & knowledge transfer

Understand the three loops, IPO evidence, portfolio decisions, profit and loss, scheduled work, and the changes still ahead.

Edition 2026-10-11

Code-grounded explanations for engineering and product teammates.

Human testing duties are a separate document.

Source: docs/TECHNICAL_DESIGN.md · SHA-256 5ea8541652cd

Inside this guide

1. Reading the evidence

2. Product map and three loops

3. Runtime, configuration and storage

4. Research: input to recommendation

5. Learning: forecast, review and memory

6. Portfolio: advice and virtual execution

7. Profit/loss and the different marksheets

8. Discovery and IPO functionality

9. Scheduled jobs and event hooks

10. Delivery, interface, identity and operations

11. Planned redesign

12. PI-2026-09 roadmap

13. Validation and KT sequence

Edition: 2026-10-11 · **Audience:** engineers and teammates learning the product

Code inspected: `954305a37cd5ea191e9039f28d171f390e325906`

PDF: StockAgent-Three-Loops.pdf, generated from this Markdown.

This document describes how StockAgent works at the revision above. It covers the three product loops, the contracts between components, the records they keep and the jobs that run them. Production state is dated and kept in one place (section 1), apart from the description of the code. Work planned in PI-2026-09 is collected in sections 11 and 12. Where another section mentions it, the text is labelled **Planned**. `check_kt_docs.py` fails if a linked source file or a documented job ID is absent at the inspected revision.

The separate human testing guide divides practical testing duties among the team. It is not a human-approval workflow inside the product.

1. Reading the evidence

Label	Meaning in this document
Current code	Traced at the inspected revision. Flags, inputs and runtime data decide whether a path actually runs.
Locally checked	Tests run in the hermetic suite or an isolated copy (section 13). A passing test describes code behaviour; on its own it does not establish a financial contract.
Production observation	Dated, read-only evidence from the deployed service: logs, the RL monitor, the watchdog's mail, or the owner's checks.
Planned	Intended behaviour in PI-2026-09 (sections 11 and 12). Not implemented.

The September audit is the baseline record of the label, timing, health, weight-bound and operational defects that this design addresses. Adaptive state exists; improved prediction performance has not been demonstrated. Writing this document triggered no production job, backfill or notification.

Production state at this edition

Production runs the inspected code. The deployment of 11 October 2026 carries it, confirmed by the owner's read-only check of the deployment list and `/health`. The table gives the configuration that differs from the repository and the state of each operational control.

Area	State in production	Evidence
Learning mode	`observe` since 2026-09-26, set by the Railway variable `RL_LEARNING_MODE`; the repository default is `adapt`. Weight versions stay unchanged across reviews, and every review records its proposal. Every envelope since the 2026-10-01 monthly forecast was issued in `observe`.	Verified 2026-10-01 (activation record)
Decision gate	`record`, the repository default: computed and logged everywhere, nothing withheld. The 2026-10-01 decision kept `record`, because 25 of 78 analyses (32%) would have abstained, most for reasons since fixed.	Enforce decision

Area	State in production	Evidence
Instrument identity	Since 2026-10-01, TATAMOTORS is disabled in the managed list, with its history kept, and TMPV is managed as its own automobile ticker. The registry lists TATAMOTORS and HEXAWARE as unresolved.	Owner's dashboard change, 2026-10-01
Prediction stores	The ownership quarantine was applied on 2026-10-01. It moved 16 stray `automobile/<TICKER>` copies, byte-identical, into `data/prediction_quarantine/20260930T184031Z-ce3556d023d3/`. Rollback remains available.	Apply record in STATE.json
Ticker resolution and bank statements	Managed tickers resolve to themselves, and the four banks' fundamentals read `ok`. TATAELXSI's October envelope, built on a wrong resolution, was regenerated on 2026-10-05.	Verified 2026-10-02 from review logs
Sector benchmarks	Each sector's technicals are measured against its own index. Yahoo returns no data for `^CNXAUTO` or `^CNXENERGY`, so automobile and renewable-energy correlations read "unavailable".	Verified 2026-10-10 from the 7–9 Oct review logs
Listing-day closes	The nightly audit grades listing-day IPO rows: 4 graded and none skipped on 2026-10-01.	Verified 2026-10-01
Chat brief scope	Chat's brief tool reads only the caller's portfolio. The owner gets the owner's brief; a member account with no holdings is told there is no brief yet and sees none of the owner's.	Verified 2026-10-11, the owner's check with a member account
Delivery	Outbound SMTP has worked since a plan upgrade on 2026-09-21. The outbox records transport acceptance; user receipt is not observable. Whether `RESEND_API_KEY` is set was not inspected.	Section 10
Backups	The nightly archive and restore drill run. No off-site target is configured, because the owner deferred it to the end of the PI, so the watchdog sends a weekly reminder.	Watchdog mail, 2026-10-07
Watchdog milestones	The milestones that lapsed in August and September were judged and retired on 2026-10-06. The next morning's run sent no LAPSED notice.	Watchdog mail, 2026-10-07
CI	Three GitHub Actions jobs run on every push. The first run, on 2026-09-28, passed.	Section 13

The rest of the deployed behaviour is live: chat sanitizing, data-health rows, review outcome counts, the delivery contract and dossier dates. Each still awaits its recorded production observation, and STATE.json lists the pending checks.

Vocabulary

Term	Plain-English meaning
Ticker / instrument	A market symbol / the actual security it represents. A renamed or reorganized company is not automatically the same investment.
Sector / graph sector	The company's business sector / the analysis implementation selected for it. A pharma stock can use a generic graph while retaining pharma identity.
LLM / agent / dimension	Language model / component with an analysis task / one score such as fundamentals. Multiple dimension names do not establish independent evidence.

Term	Plain-English meaning
Composite / weight	Combined score / importance assigned to a dimension.
Envelope	Stored forecasts for future trading sessions, including prices, verdicts and uncertainty information.
Feedback / lesson / dossier	One review result / reusable knowledge / longer-lived company observations and questions.
Advice / transaction	A portfolio recommendation / a recorded virtual buy or sell. They are separate records.
Decision gate	The check that decides whether an analysis's evidence is good enough to act on: actionable, degraded or abstain.
Price basis	The continuous price history a holding's cost and a forecast's closes are measured on. A demerger starts a new basis; a rename does not.
Cohort / matured outcome	A defined group of predictions / an outcome whose observation period has finished.
Idempotent	Repeating the same operation does not apply its effect twice.
Cache / projection	A reusable fetched result / a secondary representation of another store. Neither guarantees freshness or authority.
PI	The planned improvement program (PI-2026-09). Story status lives in its state file, apart from this design description.

2. Product map and three loops

StockAgent combines listed-equity research, adaptive feedback, a virtual portfolio, discovery, IPO information and reports. Three connected product loops explain the main flow:

1. **Research:** collect evidence, score dimensions, combine them, check the evidence and produce a report.
2. **Learning:** compare forecasts with observations, record feedback, propose or apply weight changes and maintain lessons and company knowledge.
3. **Portfolio:** evaluate held positions, record advice, optionally execute it in a virtual account, then produce transactions, value history and summaries.



Three controls cut across the loops:

- the **decision gate** (section 4) decides whether the evidence behind a verdict, a forecast, a review or a buy is good enough to act on;
- the **instrument registry** (section 4) decides which security a ticker means on a given date, and on which price basis;
- the **learning mode** (section 5) decides whether learned state may move decisions or is only recorded.

The nightly outcome auditor measures issued advice. The watchdog checks operational milestones. Neither is the daily feedback agent. There is no general human-approval queue between these components.

Repository orientation

Location	Responsibility
API server and routes	FastAPI startup, API and streaming interfaces.
src/backend/shared	Shared orchestration, canonical schemas, prompts, settings, the decision gate and the instrument registry parser.
src/backend/sectors	Native implementations, generic graph and sector registry.
services/data	Fetchers, context assembly, caches, durable stores, backup, restore and off-site copy.
core/intelligence	Forecasting, feedback, learning mode, regimes, seasonal logic, company knowledge and prediction-store tools.
core/portfolio, core/discovery, core/ipo	Virtual money path, identity reconciliation, listed-stock discovery and IPO evidence.
core/audit, core/ops/watchdog, core/delivery	Outcome measurement, operational monitoring and delivery.
src/frontend/prototypes	Served React/JSX application, PWA, chat rendering and data adapters.

Location	Responsibility
tests	Unit, integration and contract checks behind a hermetic boundary; their presence alone is not acceptance.

Some paths are compatibility re-exports. For example, `core/config/settings/base.py` forwards to `src/backend/shared/config/settings/base.py`. Follow the import before editing; two import paths need not mean two implementations.

3. Runtime, configuration and storage

Runtime

Dockerfile uses Python 3.11 and runs `services.api.server:app` with two Uvicorn workers. It copies `src/backend/` to `/app/backend/`; local tests add `src` to the import path. The served frontend is the prototype JSX application. package.json provides development tooling; there is no compiled app frontend build step. The checked-in C# scheduler is not launched by this Dockerfile.

Both workers run startup. One binds a localhost TCP port, default 59321, and becomes the background owner. It starts RL self-heal, APScheduler, cleanup and the optional Atlas outbox drainer; the other serves API requests. This guard is container-local, not a distributed lock or continuous standby takeover. `GET /health` returns process-level `ok`; it does not verify background ownership, storage, jobs, data or delivery. **Planned:** a readiness check and background-ownership recovery (SA-029).

Configuration

The loader resolves a declared environment override, then config.yaml, then a code fallback. Not every YAML field has an environment override. Most settings are resolved at import, so editing a file does not imply live reconfiguration.

Checked-in setting	Meaning
`scheduler.enabled: false`	Local startup default; production overrides are separate.
`scheduler.feedback_cron: "30 16 * * mon-fri"`	Daily review default, interpreted in Asia/Kolkata.
`rl.hard_bind_verdict_enabled: true`	Research category follows the composite; model `final_score` remains separate.
`rl.learning_mode: adapt`	Learning switch, env `RL_LEARNING_MODE`. `adapt` lets learned weights and lesson emphasis move decisions; `observe` contains them (section 5). Any other value fails closed to `observe`. Production sets `observe`.
`decision_gate.mode: record`	Essential-data gate switch, env `DECISION_GATE_MODE`. `record` computes and records the gate everywhere and changes nothing; `enforce` withholds verdicts, forecasts, learning and new-risk buys that rest on unverified data or an unresolved identity (sections 4–6). Any other value fails closed to `enforce`.

Checked-in setting	Meaning
`config/instruments.yaml`	The instrument registry: effective-dated records of which provider symbol each listed ticker is priced from, on which price basis, and whether its identity is resolved (section 4). Evidence is needed to resolve an identity, never to quarantine one. It lists TATAMOTORS and HEXAWARE as unresolved and two plain aliases as resolved. Quarantine acts through `decision_gate.mode`.
`rl.control_lane_enabled`, `rl.scorecard_enabled`: true	Control/evaluation paths configured; not proof of valid prospective comparison.
`ipo.enabled: true`; `ipo.gmp_enabled: false`	IPO refresh and grey-market-price fetching have separate gates.
`delivery.enabled: true`; `delivery.email_enabled: false`; `delivery.push_enabled: true`	Job scheduling, transport enablement and credentials are separate conditions.
`watchdog.enabled: true`; `watchdog.prep_enabled: true`	Monitoring can perform configured prep as well as report results.
`audit.enabled: true`	Nightly outcome grading configured.

These are repository values; section 1 lists where production differs. Credentials and `.env` contents do not belong in this document or in testing evidence.

Persistent records

Records	Writer / consumers and scope
`data/predictions/<sector>/<ticker>/`	PredictionStore: envelopes, feedback, controls, weight memory, lessons and dossier, plus `<TICKER>_weight_observations.json` (observe-mode proposals). RL writes; advisor, evaluation and UI read. A ticker's history belongs to `<managed sector>/<TICKER>` (below).
`_shared_ledger.json` per sector; `_market_ledger.json` at prediction root	Shared lessons can affect more than one ticker; bad attribution can propagate.
`data/managed_tickers.json`	The managed list: each scheduled ticker's sector and enabled flag. Disabling a ticker keeps its history; removing it deletes the history.
`data/prediction_quarantine/<migration>/`	Stores moved out of the live tree by the ownership quarantine, with their `lineage.json`. Inside `data/`, so the nightly backup keeps them; outside the tree the evaluators walk.
`data/portfolio/<user>/`	PortfolioStore: holdings/cash, advice and transaction JSONL, value history, dated digests, briefs and weekly reviews.
`data/ipo/`	Historical issue facts and captured demand snapshots; reports derive from these records.
`data/market_cache/ipo.json` and EOD parquet	Cached IPO information and exchange price history; date, source and stale state matter.
`data/logs/data_health.jsonl`; `data/logs/decision_gate.jsonl`	One data-health row per analysis run, also written to `telemetry.db`; one gate row per consumer decision. The gate log never holds a user id.

Records	Writer / consumers and scope
SQLite score, log, user and Atlas stores	Historical scores, telemetry, sessions and relational indexes/outbox. Secondary tables may lag primary records.
`data/scheduler_job_outcomes.json`; `data/watchdog_state.json`	Operational results, including the daily review's per-ticker outcomes, and milestone state. Not proof of financial performance.
`data/backups/`	Local rotation of the last 7 archives, each with its manifest, and `backup_status.json` (the last restore drill and off-site result). An encrypted copy goes outside the volume once a target is configured (section 10). A local archive is not off-site recovery.

PortfolioStore's JSONL ledger is the authoritative advice source. The September audit found incomplete or stale optional Atlas projections. An empty projection does not prove no advice was issued.

Prediction-store ownership

A ticker's forecasts, reviews and learned state must live in one store, or its history splits across graphs that score different dimensions.

The owner rule. A ticker's history belongs to `<managed sector>/<TICKER>`: the store the scheduled forecast and review use, taken from the managed entry's sector whether or not the ticker is enabled. Some tickers have no owner: one without a managed entry, one whose entry has no sector, and one whose entries disagree. Nothing guesses a sector.

Why the rule exists. SUZLON is managed as `renewable_energy`, and its forecasts and reviews live in `renewable_energy/SUZLON`. The startup self-heal in the server once looked for every managed ticker's envelope in `automobile/<TICKER>`. For a non-automobile ticker it found none, so on the first deploy of each month it ran the automobile graph into that directory. The 10 September audit found 15 such pairs, and all 58 feedback logs said `automobile`, because the store created each new log with the schema default.

Writers. Every writer keeps to the owner rule:

- self-heal uses each managed entry's sector, so a restart rebuilds a missing envelope in the ticker's own store;
- PredictionStore stamps its own sector on the feedback logs, weight memory and ledgers it creates. Files already on disk keep what they say, so a default can still be told from a declared sector;
- the daily review rebuilds a missing or unreadable weight file from its sector's graph, as the forecast does.

The inventory (store_inventory). `python -m core.intelligence.rl.stores.store_inventory --out manifest.json` is read-only. It reads files directly and never constructs a PredictionStore, whose constructor creates its directory. For each store it records its identity (path, ticker, file count and one digest), its sector evidence, its ownership, its schema markers and the code that reaches it. The sector status says how far the store's own data supports its directory:

Status	Meaning
`confirmed`	Every file with a dimension roster names one graph, and it is the directory's graph. A directory name alone never confirms.
`conflicting`	A file names another graph or several graphs, or declares another sector than its directory.

Status	Meaning
`directory_only`	A sector directory whose files carry no roster: only the directory name.
`roster_only`, `missing`	A legacy flat `<TICKER>/` store with, or without, a recognisable roster.
`empty`	No files, typically a directory created by a read.

A dimension roster is the set of dimension keys in a file's rows or weights. It identifies the graph that produced them: the nine automobile dimensions differ from the six renewable-energy ones. `roster_vs_owner` compares the rosters with the owner's graph. `automobile/SUZLON` is `confirmed`, because the automobile graph did produce it, and `wrong_roster` for its owner. The `sector` fields of feedback logs, weight memory and ledgers are reported, never used as evidence. Each roster comes from the graph's weight table through the sector router. If a native sector is switched off, it takes the generic roster and its own files read `unrecognized`; the summary's `roster_collisions` then names the graphs whose files cannot be told apart.

The manifest also lists duplicates, each marked identical or conflicting:

- tickers with more than one non-empty store;
- envelope cycle ids found in more than one store;
- graded days (ticker and date) recorded more than once, in one store or across stores.

These are derived keys, not issuance identities. The manifest holds paths, tickers, sector keys, counts, SHA-256 digests, dimension names and field names, and no prices, text or user data. Its digest excludes the generation time, so an unchanged tree always gives the same digest.

Who reads which store. Each rule below picks a store for a ticker:

Class	Store it picks	Code
`managed`	`<managed sector>/<TICKER>`	The monthly forecast and daily review (scheduler), startup self-heal, event ingest, weekly ledger cleanup, RL monitor and scheduler API.
`analysis_graph`	`<graph sector>/<TICKER>`	An analysis's learned-weight read (base_orchestrator) and its agents' prompt-enhancement and dossier reads. For a pharma ticker that is `generic/<TICKER>`, while its RL store is `pharma/<TICKER>`.
`automobile_default`	`automobile/<TICKER>` when no sector is passed	Manual CLIs (`run_schedule forecast` and `daily-review`, `generate_forecast`'s own main), the bundle builder's and base agent's fallbacks, the UI's last fallback.
`discovery`	every `<dir>/<dir>` pair	Scorecard, eval harness, learning evidence, analytics and universe walk the whole tree; the UI scans it for an unmanaged ticker.
`flat_cli`	`<TICKER>/`	`run_schedule feedback-status`. Discovery misreads such a directory as a sector.

The pre-open shock check follows none of these rules: on a shock morning it scans the four native sectors' built-in ticker lists, not the managed list, and reads `<list sector>/<TICKER>`. A managed ticker outside those lists is not checked, and a listed ticker without a store gets an empty directory. The inventory's own reader table still files the check under `managed`.

The last three classes, and the constructor's directory creation, remain. **Planned:** one frozen `store_key` replaces them (SA-026), and row-level lineage marks rows from another graph inside an owner's store (SA-017).

The quarantine (store_migration). It has three commands:

- **`plan`** is read-only. It proposes to quarantine each non-empty store of a managed ticker that is not the owner's store, including a legacy flat one. It holds, and never moves, anything that needs a decision: duplicates of an unmanaged ticker, an owner that disagrees with the ticker map, a managed entry with no sector or two conflicting entries, and two owner directories that differ only in case. It flags an owner's store that carries another graph's roster. The plan lists every file's SHA-256 and ends with one digest.
- **`apply --plan … --approve <digest>`** recomputes the plan and refuses unless it is unchanged. It creates a new migration directory `prediction_quarantine/<UTC time>-<digest>/` beside `data/predictions` and writes the lineage record before the first move. It re-hashes each store immediately before moving it and stops at the first difference. A store moves by one directory rename: its bytes are never copied, rewritten, merged or deleted, and they are hashed again after the move. The quarantine keeps the live layout, so `PredictionStore(ticker, sector, base_dir=<migration>/stores)` still reads it.
- **`rollback --migration DIR`** renames each store back after checking its bytes. It never merges: it refuses a live path that holds any file, and a quarantined store whose bytes changed. An empty directory at the live path, which any read through `PredictionStore` creates, is removed first. A refused store is retried on the next run, so an operator can repair it and run rollback again. A store already back at its live path with its recorded bytes counts as restored (`found_live`), whether it was moved by hand or by a rollback interrupted before it recorded the move. A run reports `rolled_back` only when every store is back with its recorded bytes; otherwise it reports `partial`, and `lineage.json` gives each store's state. After a complete apply, rollback restores the live tree byte for byte.

A racing writer is detected, not prevented, so an apply runs in a job-free window and needs the owner's authorization. Production's one apply is in section 1.

4. Research: input to recommendation

Trace: registry → base orchestrator → bundle builder → unified analyst → aggregator → decision gate.

1. Resolve input to a company, symbol and sector. A ticker the app already tracks resolves to itself with no model call (below); other text goes to an LLM, with a web search when its answer cannot be verified.
2. Select the graph. Automobile, banking/BFSI, IT and renewable energy have native implementations; other sectors use generic analysis. The RL router delegates graph selection to the central registry.
3. Assemble company/sector news, fundamentals, technicals, macro, flows, peers and company knowledge. NSE prefetch data is shared. Each section reports a typed status: it may be verified, cached, stale, a fallback, empty, not applicable or failed.
4. The preferred unified analyst scores the sector dimensions together. The legacy per-dimension pool remains a fallback. A whole research request can involve resolution, aggregation, retries and extra provider calls; it is not necessarily one LLM call.
5. The aggregator combines weighted scores, excludes errored contributions and normalizes surviving weights. With no surviving weight it falls back to a neutral composite. An LLM produces narrative and report fields.
6. With hard-bind enabled, configured score bands determine the categorical verdict. The raw model verdict is logged for comparison; its numeric `final_score` remains separate.

7. The decision gate labels the report actionable, degraded or abstain before anything logs or returns its verdict.
8. Score history, run summaries, logs and the data-health row support the user report and later diagnosis.

Example: scores 0.8 and 0.4 at weights 0.75 and 0.25 give composite `0.8 × 0.75 + 0.4 × 0.25 = 0.70`. The configured band containing 0.70 gives the bound verdict. A different LLM `final_score` is a different field, not necessarily an arithmetic error.

Data health

Every unified run writes one data-health row, to `data/logs/data_health.jsonl` and `telemetry.db`. Every section producer returns a typed result (fetch_result.py): a status, the source, the as-of date of its newest datum, and a reason. The producer decides the status from its structured data, never from its sentence. A sentence such as "Technical data unavailable for TATAMOTORS" or "[No results for: …]" is not evidence of data. A status read from such text once recorded two runs against a price source answering HTTP 404 as `ok`, with 10 live sections and 9/9 dimensions (2026-08-26).

- `ok` and `cache_hit` mean verified data. `stale` means older than the freshness bound: a newest price bar over 7 days old, or a newest reported quarter over 200 days old. `fallback` means a core figure was replaced by search snippets or substituted zeros, or the newest listed quarter has no figure (yfinance lists an announced quarter as NaN before its results are filled in; `as_of` is then the newest quarter with figures). An older quarter without a figure is named in the reason only. `empty` means nothing came back; `n/a` means deliberately not fetched; `failed:<Type>` means an exception. `unverified` means plain text with no typed result.
- A row is `ok` only when every dimension scored and every applicable section is verified and fresh. Otherwise it is `degraded` or `hollow`, and `health_reasons` says why. Fundamentals, technicals and peers valuation are also listed in `essential_unusable`, which the decision gate reads.
- A Tavily month-cache entry records how many results it held, so a cached "no results" stays `empty` for the rest of the month.
- Rows carry `contract_version: 2`. Older rows stay as written and are read as version 1 with unknown provenance, not upgraded.

Example: the price source answers 404 for TATAMOTORS while news, macro and flows answer. The analyst's prompt text is the same either way. The row says `degraded` with `essential technicals=empty`, `essential fundamentals=empty` and `essential peers_valuation=fallback`, and the decision gate reads the same statuses.

Bank income statements

A bank's quarterly income statement has interest income and net interest income, and no operating-income, EBIT or gross-profit row. The fundamentals fetcher lets the statement's shape decide how it is read, never the ticker or its sector:

- both interest rows and no operating row make a **bank-format** statement. Its net interest income takes the operating line's place: the prompt shows it per quarter, and a blank newest quarter in it still means `fallback`. The prompt says operating margin is not reported, with no 0.0 figure, and the section's reason names the format;
- revenue is still required, and the 200-day freshness bound still applies. A bank's revenue is yfinance's "Total Revenue", which is not the filing's total income: YES Bank's is below its own interest income;

- PAYTM reports EBIT beside its net interest income, so it stays a standard statement. A non-bank statement without an operating row stays `fallback`.

Substituting zeros for the missing line would make every bank `fallback`, show an EBITDA margin of 0.0% and make the gate abstain on every bank analysis. That is what happened from 28 Sep to 1 Oct: YESBANK, IDFCFIRSTB, RBLBANK and FEDERALBNK abstained on 17 of 78 analyses (22%), although their June-quarter figures were on file.

Example: YES Bank's June quarter (revenue ₹4,655.98 Cr, net interest income ₹2,785.48 Cr) reads `ok`, so the gate does not abstain on its fundamentals. Envelopes keep the gate they were issued with, so an enforce decision counts only rows issued after a reading rule changes.

Sector benchmarks

The technicals section shows the stock's correlation and beta against an index chosen for its sector:

- the bundle and the legacy agents pass the run's sector, and `settings.SECTOR_BENCHMARKS` names its index: automobile `^CNXAUTO` (Nifty Auto), banking/BFSI `^NSEBANK` (Nifty Bank), IT `^CNXIT` (Nifty IT) and renewable energy `^CNXENERGY` (Nifty Energy). `get_peer_correlation` in fetcher.py has no default index;
- a stock in any other sector, or with no sector, is measured against the broad market, `^NSEI`, labelled "Nifty 50 (broad market)". It is never measured against another sector's index;
- returns are paired over the sessions both series have a close for. Fewer than 30 paired returns, no data for either symbol, a flat series or a failed download leave correlation and beta unmeasured. The prompt then says "unavailable" and why, and the section's reason names it. The section's status stays the stock's own: the index is context for its price history;
- the section's provenance names the index (`benchmark`), and each call logs one `[technicals] <ticker> sector=<sector> benchmark=<index>` line.

An unmeasured value is never shown as a measurement. A default of "Correlation: 0.0 | Beta: 1.0" looks like data; it once appeared on every analysis, banks and IT included, because a single automobile index served every sector and Yahoo had no data for it.

Example: HDFCBANK's technicals read "Nifty Bank Correlation: … | Beta: …", measured against `^NSEBANK`. Yahoo has no data for `^CNXAUTO` or `^CNXENERGY`, while `^NSEBANK`, `^CNXIT` and `^NSEI` each answered with 188 sessions on 5 Oct. So an automobile stock reads "Nifty Auto Correlation: unavailable | Beta: unavailable (benchmark ^CNXAUTO returned no price data)", and a renewable-energy stock reads the same for Nifty Energy. **Planned:** choosing an index with data for those two sectors belongs to the sector lenses (SA-026).

Decision gate

Every report carries a typed `decision_gate` (decision_gate.py), computed from the same section statuses and dimension counts as the health row, whether or not the row is recorded. The aggregator renormalises over whatever was scored, so without a gate one surviving dimension becomes the whole verdict: production once recorded BUY on 1 of 6 scored dimensions and STRONG BUY on 3 of 9.

- **abstain** when an essential section (`observability.data_health_essential_sections`) is not `ok`/`cache_hit`, when fewer than half the sector's dimensions were scored (`decision_gate.min_dimensions_fraction: 0.5`), when the run has no bundle at all (the legacy worker-pool fallback has no provenance), or when the ticker's identity fails (below). An unresolved symbol lands here too: no provider knows it, so its essential sections are empty.

- **degraded** when it may act but an ordinary enrichment (news, macro, flows, a dimension above the floor) is missing. Degraded is actionable.
- **actionable** when nothing is missing.

One switch, `decision_gate.mode` (env `DECISION_GATE_MODE`), governs every consumer. In `record`, every consumer computes the gate and records what enforcement would stop: in `data/logs/decision_gate.jsonl`, on the report, on forecast rows and on advice. Nothing else changes. In `enforce` an abstaining report's verdict reads `INSUFFICIENT DATA` (the aggregator's verdict is kept as `withheld_verdict`), and sections 5 and 6 describe what the learning and portfolio consumers then refuse. An unrecognised value fails closed to `enforce`.

Example: the TATAMOTORS 404 run above scores 9 of 9 and the aggregator says STRONG BUY. Its gate is `abstain`, naming the three essential sections. In `record` the user still sees STRONG BUY and the gate log gains a row saying what enforcement would have withheld; in `enforce` the user sees INSUFFICIENT DATA. The same run with its price source answering stays `actionable` and keeps STRONG BUY.

Instrument identity

The gate's section checks prove that evidence arrived and is fresh, not that it belongs to the company the ticker names. After the 2025 demerger, an undated override mapped TATAMOTORS to TMPV.NS, the passenger-vehicle company, while the name "Tata Motors Limited" passed to the commercial-vehicle company. The news and the prices could then describe two companies, and with TMPV's data healthy the analysis was `actionable`.

The instrument registry, config/instruments.yaml, records identity explicitly. `symbol_resolver.resolve_identity` reads it, and its parsing lives in instruments.py. For each listed ticker it holds effective-dated segments. Each segment names the provider symbol to ask, the **price basis** those prices are on, and a status:

- only an `active` segment resolves. When its symbol is not the ticker's own listing, it must say how the two relate. An `alias` needs a reason. A `rename`, `demerger` or `relisting` needs evidence: an exchange or company document with a reference and a date;
- `unresolved`, `suspended` and `delisted` quarantine the ticker, and need only a reason. Evidence resolves an identity; nothing is needed to quarantine one;
- the ticker is also quarantined by an invalid record, a date no segment covers, or an unreadable registry (fail closed);
- a ticker the registry does not list resolves to its own NSE listing. A learned self-heal to another code (`data/yf_symbol_cache.json`) is still fetched, but is `unresolved` until someone records it;
- a registered ticker is never self-healed. An empty fetch is an outage or a lifecycle event, and a Yahoo search could swap in another company.

A rename keeps the basis: the same company under a new code, and the provider moves its history to the new code. A demerger or a relisting starts a new basis, because the continuing company's price no longer includes what was spun off.

The gate abstains when the ticker's identity for today is not resolved, or when the technicals' price history came from another symbol than the identity names (the section's provenance records its `symbol`). Identity reasons come first, and the report's `decision_gate.identity` records the instrument (ticker, symbol, basis, status). The close fetchers ask for the resolved instrument, including the BSE fallback, and the NSE cross-check uses the registry's NSE code.

Example: TATAMOTORS is `unresolved`, still priced from TMPV.NS. With healthy data the analyst says STRONG BUY:

- in `record` the user still sees STRONG BUY, and the gate log gains a row starting "identity unresolved: retired by the owner's decision …";
- in `enforce` the user sees INSUFFICIENT DATA.

TMPV and TATAMOTORS. The passenger-vehicle company is tracked as its own ticker, TMPV, by the owner's decision of 29 Sep. TMPV needs no registry record: its ticker, its name and its prices are one company, so it resolves as its own listing, and its learning starts fresh. TATAMOTORS stays quarantined, so its history (weights learned partly on the pre-demerger company) is never graded or advised against another company's prices. TMPV is in the automobile sector's exact-match list, so the LLM ticker lookup, whose prompt names "TATAMOTORS – Tata Motors Ltd", is never asked to resolve it. A retired ticker is disabled in the managed list, never removed, because removing it deletes its prediction history.

Ticker resolution

A ticker resolves to itself, with no LLM or web-search call, when it is in its sector's `TICKERS`, in the managed list (`data/managed_tickers.json`, any sector, enabled or not) or in the instrument registry:

- the managed list and the registry are read on every resolution, so a ticker the owner adds is known at once. Nothing writes the list on this path;
- the company name comes from the curated or learned name, else yfinance for the registry's symbol, else the ticker; never from the LLM;
- a retired or unresolved ticker such as TATAMOTORS resolves to itself, and the identity gate above decides whether it may act;
- an unreadable registry quarantines every ticker but names none, so free text is not mistaken for a ticker;
- free text ("tata elxsi") goes to the LLM. So does a weekly discovery candidate that no list holds, so its shelf idea can still rest on another company's analysis.

Asking an LLM which company a tracked ticker is lets it answer wrongly. At 09:16 on 1 Oct the monthly forecast for TATAELXSI (Tata Elxsi) was resolved to TATAMOTORS: the run analysed Tata Motors, with Tata Motors' learned weights, and saved TATAELXSI's October envelope from that analysis. The same path turned TVSMOTOR into TVSMOTORS and named HAPPSTMNDS "Happy Smile Digital Ltd". **Planned:** one sector definition replaces the per-sector lists (SA-026).

Limits of the research path

- The gate records only (section 1). In the 2026-10-01 measurement, 25 of 78 analyses would have abstained: 17 were the bank statements and 2 the resolution defect above, both since fixed. A blank newest quarter still makes fundamentals `fallback`, and so abstains, for weeks after each quarter end. Identity reasons share the same switch, so enforcing the gate also quarantines TATAMOTORS and HEXAWARE. The owner re-decides on rows issued since those fixes (record).
- Bundle call counters are not complete provider or fallback cost accounting. **Planned:** SA-025 counts actual calls, before SA-026 consolidates sector definitions and SA-027 retires only justified fallback duplication.
- **Planned:** the one-engine redesign replaces the per-sector graphs with sector lenses and a deterministic factor engine (section 11).

5. Learning: forecast, review and memory

Trace: forecast generation, interpolator, daily review, feedback agent, weight adapter, prediction store, learning mode.

Forecast generation

The workflow loads weights and knowledge, runs analysis and creates an envelope across future exchange sessions. A model-shaped profile, static fallback and simulated paths supply forecast prices and bands. Generated forecast rows are not independent observed samples or guaranteed probabilities.

The monthly trigger is the **calendar first at 09:00 IST**. The generated rows use exchange trading dates. Do not call that trigger the first trading day. Shocks, thesis changes and later evidence can revise or archive envelopes.

Each forecast row records where it came from:

- `source_run_id`, the analysis run that issued it, and `data_gate`, that run's gate. The envelope keeps the latest run's full `decision_gate`;
- `instrument`, the instrument its analysis priced (ticker, provider symbol, price basis, identity status). A later registry change never rewrites it;
- the envelope's `learning_mode`.

A row issued before these stamps existed has an empty `data_gate`, meaning provenance unknown, and `instrument: {}`. For such a row the review asks the registry whether the basis changed between the envelope's issue date and the session.

In `enforce` mode an abstained analysis builds no envelope (`InsufficientDataError`; the monthly job logs it) and regenerates nothing on a shock (the current envelope stays as it is). Nothing scheduled retries a withheld month-start envelope before the next month's run: only a restart's self-heal, which checks each ticker's own store, regenerates a missing envelope. Until then that ticker's reviews return `no_envelope` and its ADDs stay blocked. **Planned:** a same-month retry (SA-036), and immutable issuance identity so that revised knowledge never replaces the earlier decision's evidence (SA-013).

Daily review walkthrough

1. The scheduler selects the previous exchange trading session.
2. Daily review finds its envelope row and obtains the actual close, including the configured verification path.
3. It checks the row's provenance, the ticker's identity and the close (below). In `enforce` mode the first failure ends the review.
4. It computes price error and the current direction label. Some small, correct misses skip a fresh analyst run.
5. Otherwise it may rerun analysis, and that run must be actionable too. With hard-bind enabled, current code can grade the old outcome against the fresh report's verdict.
6. Feedback classifies misses and attributes them to dimensions. In `adapt` mode weights change; in `observe` mode the change is recorded as a proposal. Lessons and other learning state are updated.
7. Conditional thesis review, reforecasting, dossier curation and shared knowledge updates can run under their feature gates.

8. After harvesting reviews, the scheduler records one outcome per ticker (section 9) and invokes the portfolio pipeline.

Review input checks

The review checks these inputs, in order, before it writes anything:

- the graded row came from an actionable analysis (not `abstain`, not unknown);
- **identity:** the ticker's identity on the review session is resolved, and the row's price basis is the session's. A stamped row names its basis. For an unstamped row, the registry is asked whether a demerger or relisting lies between the envelope's issue date and the session; a rename does not break the basis;
- the actual close is from the bar dated the review session. The close fetchers fall back to the newest earlier bar when the session has none (a suspended symbol, a provider lag), and the result names the bar it came from. A download that holds a single bar, as on a listing day, is read like any other;
- the fresh re-run, when one happens, is actionable.

In `enforce` mode the first failure ends the review as `data_gated`: no grading, FeedbackAgent call, weight update or observe-mode proposal, lesson, envelope revision, streak, feedback entry, dossier or control-lane step. The skip is recorded with its stage, reason and source run id. The market-wide sticky regime, updated at the start of every review, is the one write that still happens. In `record` mode the review runs and its summary lists what enforcement would have stopped. The scheduler counts a `data_gated` review on its own (section 9).

Example (identity): a row issued on 5 June predicts a parent's close, and the parent demerges on 15 June. From then on the ticker's close is the continuing company's alone, so a review on 20 June would grade a "fall" that is really the spin-off. The review stops at `identity`.

Example (stale close): a symbol is suspended on the review day, and both providers return the previous day's close of 99.0. Graded, that is a flat day, a zero return that never happened. The close says it is from the previous day's bar, and enforcement skips the review.

Known limit (planned for SA-012): when NSE has not yet listed the session and yfinance's session close differs from NSE's latest row by more than 1%, the cross-check prefers NSE's earlier close. That close is marked stale, not graded.

Grading defect: an example everyone can follow

Suppose the issue-time close was 100, the predicted close 110 and the observed close 105. The stock rose **5%**, but undershot its forecast by **4.55%**. Current `classify_direction(actual, predicted)` compares against the forecast and labels this DOWN. That measures error direction, not investment direction. A new verdict computed after observing the outcome can also become the graded verdict. Label definition and issue-time information are separate problems.

Existing tests can describe those code paths without establishing the desired financial contract. **Planned:** SA-012 defines the target; SA-013 and SA-014 freeze and grade the information available when the decision was issued.

What adaptation means here

Weights, accuracy history, lessons and dossier records change over time. Regime and lesson modifiers affect research inputs. This is an adaptive feedback system; it does not demonstrate that adaptation beats a fixed policy. The audit also reproduced final weight-bound violations and repeated ensemble trend counts presented under different agent names.

Unconstrained, the adapter drove chart weights toward zero. The 2026-09-24 baseline found `technical` at 0.0 for 5 of the 6 tickers that have that agent; the defaults are 0.12 in the generic graph and 0.10 in the renewable graph. Counting `pattern_analysis`, the chart agent of the other graphs, 9 of 18 tickers had a chart weight of 0.0 and 16 of 18 were below half their default. All 19 reviews that day wrote a new weight version.

Learning mode

`rl.learning_mode` has two values. `adapt` lets the review update live weights on every eligible review, as described above. `observe` contains learning:

- Forecasts, re-forecasts, public analysis, and the review's re-scoring and forecast revision aggregate with the sector's configured default table (the table a ticker with no learning state gets), whatever the file stores.
- The review never writes adapted weights; a missing weight file is still created from the defaults at version 0, as in `adapt`. The adapter runs on a copy, and its would-be weights, deltas and reason go to `<TICKER>_weight_observations.json`, one record per review date.
- Tagged-lesson emphasis and the older category micro-adjustment stop moving agent scores. Lessons are still recorded, and `claims_fired` stays empty because no claim acted.

Example: a ticker stores `technical = 0.0` at v41. In `observe` mode its forecast uses 0.12; after the review the file still says 0.0 at v41, and the observation record shows what v42 would have been. Switching back to `adapt` resumes the stored weights unchanged.

An unrecognised value fails closed to `observe` with a warning. The mode is shown in the review's start and completion log lines, on each envelope (`learning_mode`), and by `/ui/rl/weights/{ticker}` (with the live `decision_weights`) and `/ui/rl/summary/{ticker}`. The paper lane never trains or writes weights, and the absurd-price-error guard skips the adapter; like every other decision path, the paper lane uses the sector defaults in `observe`. The mode does not contain regime multipliers, thesis multipliers, the conviction streak, miss-counter prompt enhancements or dossier text.

Production runs `observe` (section 1). Rollback is deleting the Railway variable. In this mode each night re-proposes one step from the frozen stored file, so the observation records do not accumulate into a learner. **Planned:** a shadow learner, and the conditions under which learning may return to `adapt` (section 11).

Company knowledge

Event ingestion adds company observations; the curator maintains knowledge and questions; the research job attempts answers. These are knowledge tasks, not trades. Discovery paper tracking uses a separate store root with learning writes disabled for that lane; it is not live investor performance.

Dates. No writer dates a dossier row after the run that wrote it. Event ingestion dates rows with the event's day, and skips an event dated after the scan, such as a board meeting announced for later in the month, until a scan on or after that date. The digest the agents read leaves out observations and open guidance dated after its own date; stored rows are not rewritten. Older rows can still carry a future date: on 7 Oct two production dossiers held 3 observations and 1 guidance item dated 24 and 27 Oct (probe).

Open guidance is read through one rule, the dossier's `open_guidance(as_of)`: open items dated on or before `as_of`, in stored order. The digest lists the last 5. The morning brief's earnings-watch line takes the last item as of the day before the brief's date. The brief goes out at 08:50, before any weekday dossier writer runs (the curator runs inside the 16:30 review; event ingestion and research run on Saturdays). So a row dated the brief's own day can only be an old future-dated one, and the brief's earnings window includes the meeting day.

Example: a held stock has results on Tue 27 Oct. Its dossier holds open guidance dated 15 Sep and an old item dated 27 Oct. The briefs of Mon 26 and Tue 27 Oct, the two in the 3-day earnings window, both show the 15 Sep item. The cost of the rule: a brief triggered by hand after the 16:30 review leaves out that day's new guidance until the next day.

6. Portfolio: advice and virtual execution

Trace: pipeline → advisor → executor → store → digest.

The pipeline loads active users' portfolios, obtains closes and signals, evaluates held positions, records advice and optionally executes it. The LLM narrates the rule-driven decision; triggers remain available when narration fails.

Research verdict	Portfolio action
STRONG BUY, BUY, NEUTRAL, SELL, STRONG SELL	ADD, HOLD, TRIM, EXIT, SWITCH
Assessment of a stock at analysis time	Action for an existing holding, using entry cost, risk, position size and evidence

A BUY report does not automatically purchase shares. A portfolio HOLD does not mean a flat forecast.

Advisor precedence

The cascade is **EXIT** → **TRIM** → **ADD** → **HOLD**. EXIT includes stop breach, bearish thesis/shock conditions, crisis conditions and an armed trailing stop. TRIM considers profit with declining confidence/reversion. ADD requires bullish, sufficiently healthy conditions and position headroom. EXIT may become SWITCH when an eligible replacement exists. A holding-age rule can soften certain TRIM actions to HOLD; it does not override EXIT. These are product rules, not individualized tax or investment advice.

Data and identity checks on advice

New risk needs verified evidence. ADD and a SWITCH's buy leg add risk, so in `enforce` mode they need two things: the holding's close must be from the review session's own bar, and the forecast rows behind the signal must come from an actionable analysis. A blocked ADD becomes HOLD. A blocked SWITCH destination falls back to the next eligible idea, or to a plain EXIT. A shelf idea without gate provenance, from an abstained deep dive, or priced from an earlier bar is not a destination. Either way the advice gains the note `DATA_GATE` and a `data_gate` record (what was withheld, why, and the source forecast runs). That record is on the user's own advice ledger; the shared gate log never holds a user id. EXIT and TRIM are never blocked by data: missing data must not trap a position. When they rest on a stale close or an unverified forecast, the advice records that. In `record` mode the decision is unchanged and `data_gate` says what enforcement would have withheld.

Example: a holding's envelope was built on an abstained STRONG BUY and points up. Unenforced, that gives ADD and a virtual buy of 2 shares. Enforced, it gives HOLD with `DATA_GATE`, and no buy.

Identity hold. A holding is held outright, EXIT and TRIM included, with the note `IDENTITY`, in two cases:

- its symbol's identity is unresolved;

- its price basis changed (a demerger or a relisting) since the basis its cost was booked on: the buy date, or the latest identity reconciliation.

This is the corporate-action rule applied to identity. The daily sync keeps a 1:1 bonus from looking like a 50% crash, and a demerger is the same trap. A SWITCH destination or an autopilot buy leg with an unresolved identity is refused.

Example: 100 PARENT bought at ₹1,000; PARENT demerges into PARENTA (60% of the cost) and PARENTB (40%), and the ticker now trades as PARENTA at ₹600. Unreconciled, 600 against 1,000 reads −40%, a stop breach, and the advisor says EXIT, a sale on a loss that never happened. Enforced, it says HOLD with `IDENTITY`; in `record` it keeps EXIT, and `data_gate` names the identity. A rename keeps the basis, so a renamed holding is advised as before. Every advice record also keeps the instrument its close priced (`instrument`), which a later registry change does not rewrite.

Identity reconciliation

Reconciliation is an operator's step (identity_reconcile). `python -m core.portfolio.identity_reconcile plan` is read-only. It lists every held position the advisor would hold, with the registry's proposal. The proposal comes from a recorded, evidenced event with `successors` (ticker, shares per old share, share of the cost), or it says what must be recorded first. For the example it proposes PARENTA 100 @ ₹600 (₹60,000) and PARENTB 100 @ ₹400 (₹40,000), so the cost is conserved. `apply --plan … --approve <digest>` has these safeguards:

- it refuses unless a fresh plan has the approved digest, so nothing changed since review;
- it copies `portfolio.json` aside before writing;
- it replaces the old holding with its successors under the user's lock, and logs `identity_reconciliations.jsonl`;
- a second apply is refused.

The ledger reconciler treats the successors as `unverifiable`, as after a split. Nothing runs this on a schedule.

Known limits of `apply`. Its digest check runs before it takes each user's lock, and under the lock it does not look at the holding again. So a sale of the same holding in that short window still gets successors sized from the plan's figures: 50 PARENT left would become 100 PARENTA and 100 PARENTB. Also, one apply names every backup with the same timestamp. For a user with two reconciled holdings, the second copy overwrites the first, so the only backup holds the half-applied portfolio; the audit log's `before` records still hold each old holding. The shipped registry records no `successors`, so `apply` applies nothing today. **Planned:** a fix for both (SA-008 change 1). Until it lands, record no `successors` and run no `apply` in production.

Execution and persistence

The global gate, user's virtual-autopilot choice and cash-accounting state all matter. Execution reloads the portfolio under a lock, sells before buying, and uses transaction IDs plus a monotonic review-date marker. Future and already-completed dates are rejected/skipped. Cash, sizing and eligibility can prevent a recommended action from becoming a transaction.

Transactions are appended before saving holdings/cash. A crash in between can leave the ledger ahead of the portfolio; deduplication is not an atomic multi-file transaction. Reconciliation handles this gap. Corporate actions affect adjusted quantities and cost. No broker-order execution path was found. In `enforce` mode the executor also re-checks a SWITCH buy leg's own price: if its close is not from the session's bar, the sell executes and the buy is skipped, and the user gets the existing `switch_buy_skipped` alert.

Cash deployment

The autopilot opens a new position only through a SWITCH, putting one sale's proceeds into a shelf idea. ADD tops up a stock already held, capped at `advisor.max_position_pct` (10%) after the trade. A plain EXIT leaves its proceeds in cash, and nothing reinvests them. The SWITCH buy spends the whole proceeds, with no position or sector cap.

Example (the owner's screen, 27 Sep): ₹8,09,297 of ₹9,99,611 was cash (81%), with two holdings. ACMESOLAR's `exit_full` alone moved about ₹2.27L to cash, and FEDERALBNK was 13.1% of the portfolio.

Planned: one sizing rule for every autopilot buy (SA-050). In a normal market, idle cash goes into shelf ideas that pass the checks; cash is held on purpose only in a crisis (SA-049, the owner's decision of 2026-09-27, behind a switch that ships off).

7. Profit/loss and the different marksheets

“Marksheet” covers several distinct reports. Each needs its own question, dates, denominator and exclusions.

Virtual P/L

portfolio_api.py exposes holdings, transactions, advice, digest and `/portfolio/performance`.

Measure	Meaning
Holding market value	Adjusted quantity × marked price.
Unrealized holding P/L	Adjusted quantity × (marked price − adjusted average cost).
Realized P/L	Ledger gain/loss on quantities actually sold virtually.
Total equity	Remaining holdings' market value + deployable cash.
Total return	`(total equity - capital_in) / capital_in`, when required values exist.
As-of date	Date of the valuation, potentially different from page-open time.

The virtual P/L charges no brokerage, STT, stamp duty, exchange fee, DP charge or tax. A real round trip costs about 0.25% of the traded value, and short-term gains are taxed. For ACMESOLAR's +₹37,551, that is about ₹477 in costs and about ₹7,415 in tax. The screen's "Invested" is `capital_in`, the mock money ever put in, not the money in stocks. The screen has no benchmark. **Planned:** costs and tax in paper P&L (SA-048), and a Nifty comparison with honest labels (SA-051).

Example without fees/corporate actions: start with 2,000 and buy ten units at 100. Cash is 1,000. At 110 the holding is 1,100, equity 2,100 and unrealized P/L 100. Sell four at 110: cash becomes 1,440, remaining holding value 660, realized P/L 40 and remaining unrealized P/L 60. Equity stays 2,100.

Current performance reads at most 2,000 transactions and 400 history rows. With no history, a failed price lookup may fall back to adjusted purchase cost. The digest sums priced holdings only, and its `portfolio_value` excludes cash. These scope differences need interpretation; a displayed total is not proof all prices are fresh or all history is included. These virtual results do not establish executable net returns after fees, spread, tax and slippage.

Report families

Report	Present meaning	Limitation / planned work
RL Monitor / analytics	Envelope, feedback, miss and weight records, with the learning mode and live decision weights.	Inherits label/timing/history problems; SA-012–SA-017, SA-024.
Weekly scoreboard	28-calendar-day lookback, non-HOLD calls at least five calendar days old, latest supplied closes.	Not fixed matured horizons; current-holding price selection can omit exited stocks. SA-018/SA-019.
Monthly scorecard	Agent/control/simple-baseline lanes and previous-month comparisons.	Completeness, timing and denominators need repair. SA-018.
Learning evidence	Retrospective adapted/base/uniform replay and lesson associations.	Not a prospective controlled experiment of the whole system. SA-020–SA-023.
Nightly advice auditor	Issued advice, alert, shelf, switch and IPO lanes; default 10/30/60 trading-session horizons, and 1/5/21/63/126/252 from listing on the IPO lane.	Separate from weekly heuristics and actual account P/L. IPO rows are excluded from the rendered report.
IPO history report	Subscription groups and issue-price returns at available horizons.	Descriptive associations with sample/feature gaps, not a validated prediction model.

Audit rules judge HOLD/ADD correct when excess return over the benchmark is nonnegative, and TRIM/EXIT/SWITCH correct when negative. The switch-pair lane separately asks whether destination beat origin over the same dates; a tie does not establish benefit. An EXIT can score correctly while the actual virtual trade realizes a loss: these measure different things.

The IPO lane (grade_ipo_lane) grades the P3 IPO verdicts under `is_ipo_correct`, a separate rule from the advice vocabulary. Its entry price is the **issue price**, not a market close, and its horizons count trading days from listing with horizon 1 being the listing day itself. Only one row per issue can carry a True/False: the listing-day horizon, and only when the verdict was taken after the book closed and its lean asserted a direction. Every other horizon is recorded with `correct` unset, so it enters no hit-rate. These rows are written to the same store and excluded from the rendered audit report: the P3 model remains dark until its visibility gate is met, and a displayed hit-rate would be that surface.

On a listing day a download holds one bar, and the close fetcher reads it like any other. So the listing-day row, the one per issue that can carry a mark, gets its close. A row that gets no close is not written, so the next nightly run retries it. **Known limit (planned for SA-012):** the audit prices every row through `close_on`, which returns the close without its bar's date. When a provider lags, a close carried forward from an earlier bar is stored as the session's. No bar precedes a listing, so the listing-day row is not exposed.

Example: stock +5%, benchmark +8% means excess **−3 percentage points**. HOLD is incorrect under that relative-return rule despite a positive stock return. Every marksheet should identify its rule, horizon, matured sample count, missing data and excluded cases.

8. Discovery and IPO functionality

Discovery

run_discovery_cycle connects EOD sync, bulk/block activity, optional recent-IPO candidates, screen, deep dives, conviction shelf and paper reviews. Failures can degrade individual stages. The Saturday job and manual discovery endpoint use this entry point. A shelf candidate, watchlist promotion and virtual purchase are separate steps. A deep dive runs the ordinary analysis, so its decision gate, including the identity check, is the shelf idea's `data_gate`. In `enforce` a candidate with an unresolved identity is never shelved, nor used as a SWITCH destination.

IPO layers

Layer	Current source and output
Calendar, offer and bids	ipo.py, ipo_offer.py, ipo_bids.py: normalize and enrich issues. Failed refresh preserves previous cache with degraded/stale information.
Captured snapshots	signals.py: observed refresh facts with hour/content deduplication. velocity.py: demand changes derived on read.
Historical evidence	history.py, outcomes.py, report.py: facts, realized curves and subscription-bucket summaries.
P3 model and deep dive (dark)	research.py, extract.py: browsed, corroborated Substance facts with source URLs. hype.py, substance.py, verdict.py: deterministic indices and the §3 verdict grid over captured and browsed facts. deep_dive.py: the 19:00 `ipo_deep_dive` sweep (T−1 research run, post-close re-read from cache) that writes to the append-only store in verdicts.py. narrate.py: a sourced research note stored on that row, written by the bulk model from the SAME structured findings the indices read — the verdict is never passed to it, every number in the prose must appear in the findings, an advice/verdict vocabulary rejects it, and a rejected or failed note falls back to a deterministic template. **Writes only:** no verdict, index or quadrant reaches any surface until the `ipo_verdicts_visible_gate` milestone is judged on forward rows.
Forward grading of the P3 verdict (dark)	listing.py resolves the listing date and issue price from the P1 spine, then the NSE cache; grade_ipo_lane grades the newest stored verdict per issue against the tape at 1/5/21/63/126/252 trading days from listing, entry price = issue price. Only the listing-day row of a post-close verdict with a directional lean can be scored; the rest carry the return and no claim. Rows land in the existing per-user audit store and are **excluded from the rendered audit report**. `scripts/ipo_backfill.py` still rebuilds the P1 spine manually.
Recent-listing ranking	ipo_tracker.py: listing evidence, delivery trend, bulk accumulation and optional subscription score discovery candidates.
User surfaces	IPO-watch in briefs, weekly/discovery context and shelf. The brief's demand lean (`_ipo_lean`) judges subscription against size-tiered bands from `delivery.brief_ipo_size_tiers`; an issue whose size was not read uses the scale-free scalar thresholds, never the largest tier. The lean is a labelled heuristic, not the P3 verdict. Inspected routes do not provide a dedicated `/ipo/predict` API or complete standalone IPO prediction page.

History uses **1, 5, 21, 63, 126 and 252 trading sessions**, measured from **issue price**. Unmatured horizons are absent, not zero. Benchmark excess subtracts the listing-to-horizon benchmark return; that window does not capture the stock's issue-to-listing move in the same way. Do not interpret the result as a fully matched investable strategy. Missing subscription is excluded, and report bucket means below a sample count of ten are suppressed.

The recent-listing tracker is a separate heuristic. It needs at least five EQ sessions and configured price/liquidity guards. Component weights are 40% listing evidence, 20% delivery trend, 15% bulk accumulation and 25% subscription, renormalized over available components. It can fall back to first cached close when issue price is absent; that differs from historical-outcome calculation.

GMP fetching exists behind a separate disabled-by-default gate. Missing GMP does not mean zero. Lock-in alert dates use fixed 30/90/180-calendar-day offsets from listing; these are software heuristics, not verification of an individual issue's contractual dates.

The historical **Prospect** design planned further modeling after P0/P1/P2 collection. Current code has collection, history, captured signals, heuristic candidate ranking and the dark P3 model, deep-dive job, narrator and forward-grading lane in the table above. None of it reaches a user. Production's nightly audit grades IPO-lane rows (4 on 1 October 2026; section 1), and they stay out of every rendered report until the visibility gate is judged. A validated IPO application/allotment or listing-gain predictor is therefore still not established, and is not part of the September remediation scope.

9. Scheduled jobs and event hooks

Source: scheduler.py. These are in-process APScheduler jobs, not separate OS cron jobs. There are **24 possible job IDs**, including three IPO jobs. Registration depends on gates and valid cron configuration; registration alone is not execution. All times are **Asia/Kolkata (IST)** defaults.

Job ID	Schedule	Work / condition
`prompt_daily_deploy`	Daily 00:00	Prompt publishing; handler/configuration can write externally.
`ops_watchdog`	Daily 06:30	Milestones/checks/configured prep; watchdog gate.
`macro_daily_news`	Mon–Fri 07:30	Policy/RBI feed; macro-news gate.
`ipo_refresh_am`	Daily 08:00	Calendar/bids/snapshots; IPO gate.
`preopen_shock_check`	Mon–Fri 08:45	Shock/reforecast; feature/calendar checks in handler.
`morning_brief`	Mon–Fri 08:50	Brief assembly/delivery; delivery gate.
`macro_market_news`	Mon–Fri 09:00, 12:00, 15:00	Intraday macro feed; macro-news gate.
`rl_daily_review`	Mon–Fri 16:30	Previous trading session; configured feedback cron.
`ipo_refresh_pm`	Daily 17:45	Evening IPO update before Sunday weekly review; IPO gate.
`ipo_deep_dive`	Daily 19:00	T−1 IPO research + post-close re-read into the dark verdict store; IPO gate.
`bhavcopy_daily_sync`	Mon–Fri 19:00	Official EOD cache; discovery gate.
`atlas_universe_recompute`	Daily 23:00	Demand/cadence; no-op if Atlas disabled.

Job ID	Schedule	Work / condition
`atlas_cost_rollup`	Daily 23:15	Cost aggregation; Atlas gate in handler.
`atlas_retention`	Daily 23:20	Retention; Atlas gate in handler.
`data_backup_nightly`	Daily 23:30	Archive with manifest, restore drill, local rotation, encrypted off-site copy when configured, email copy; the status feeds the watchdog (section 10).
`audit_nightly`	Daily 23:45	Matured outcome grading/breach reports; audit gate.
`ledger_cleanup_weekly`	Monday 03:30	Stale-lesson cleanup.
`event_ingest_weekly`	Saturday 10:00	Dossier events; event-ingest gate.
`research_loop_weekly`	Saturday 11:00	Open company questions; research-loop gate.
`discovery_weekly`	Saturday 12:30	Screen/deep dives/shelf/paper; discovery gate.
`weekly_review`	Sunday 18:00	Portfolio summary/index watch; delivery gate.
`scorecard_monthly`	Calendar first 02:00	Previous-month scorecard/evidence; scorecard gate.
`rl_monthly_forecast`	Calendar first 09:00	Forecast envelopes.
`rl_calendar_update`	December 31 23:00	Holiday calendar refresh.

Startup self-heal, the post-review portfolio/digest hook and the outbox drainer are **not additional cron jobs**. An 08:50 brief is not guaranteed to wait for an unfinished 08:45 shock check. Close clock times are not dependencies. HTTP 202 from a manual trigger means accepted for background work, not finished.

Daily-review outcomes

A review that does not raise has not necessarily produced feedback. Counting it as output once recorded `produced=20`, `expected=20` and `pipeline_ok=true` on a day when WELCORP returned `no_envelope` and only 19 feedback rows existed. review_outcomes.py therefore gives each enabled ticker exactly one outcome:

- `completed`: the review wrote its feedback entry;
- `degraded`: it wrote its feedback entry on an input the data gate would have stopped, as `record` mode lists them. A graded row with unknown provenance reads `degraded`;
- `data_gated`: the gate stopped it before any write (`enforce` mode);
- `skipped`: no envelope, no forecast row for the session, or no close;
- `failed`: it raised, returned a malformed result (an unknown status, or a result for another ticker or date), or was still running when the harvest budget ran out.

Only `completed` and `degraded` are output (`produced`), so a skip, a gate stop or a failure fires the zero- or partial-output alert, which names the tickers and why.

Example: 20 tickers, and WELCORP has no envelope. The record reads 19 of 20, `status: partial` and `missing: {WELCORP: "skipped: no_envelope"}`. The alert says "completed 19/20 — missing: skipped no_envelope: WELCORP".

- Required work (the distinct enabled tickers) and attempted work are counted separately. Disabled tickers are listed as `excluded`, and a duplicate entry is reviewed once and listed under `duplicates`.
- The day after a weekday NSE holiday reviews the same session again: Fri 2 Oct and Mon 5 Oct both review Thu 1 Oct. The second run merges with the first, so attempts add up and completion does not. A ticker whose earlier run wrote its feedback stays output even if the rerun fails.
- `status` is `ok`, `partial`, `failed` or `empty` (every ticker disabled). `pipeline_ok` is true only when the portfolio pipeline returned a well-formed `completed` and no required review failed; `pipeline_status` says what the pipeline itself did.
- The record in `data/scheduler_job_outcomes.json` keeps the legacy fields and adds `by_ticker`, with each ticker's outcome, reason and attempts.

Limits. The pipeline's own `completed` does not count holdings whose advisor call failed inside it. The learning side of a rerun is not yet idempotent. Nothing in the watchdog reads this record. **Planned:** durable outcomes for every critical job (SA-036), idempotent learning updates (SA-015), silent-degradation checks in the watchdog (SA-034), an operations backlog (SA-011), and readiness and ownership recovery (SA-029).

10. Delivery, interface, identity and operations

Delivery

Briefs, weekly reviews and digests persist independently of transport. Channels send email and web push. Email has two transports: SMTP (STARTTLS) and the HTTPS Resend API. `EMAIL_TRANSPORT` in settings defaults to `auto`, which uses Resend only when `RESEND_API_KEY` is set and SMTP otherwise; `resend` or `smtp` forces one. The repository's `delivery.email_enabled` is `false`; the `DELIVERY_EMAIL_ENABLED` environment variable overrides it.

When its Atlas path is active, outbox.py holds one row per message and channel, and a drainer in the singleton owner sends each row. Stored, queued, transport-accepted, received and read are separate states; the code observes only the first three. A row's stored status `delivered` means the transport accepted it.

Delivery contract

Every transport returns one result: whether it accepted the message, why not, whether a retry can help, and any wait the provider asked for. The drainer acts on it:

- **Accepted is not received.** SMTP 250, a Resend 2xx and a push service's 201 are acceptance. The row records the transport in `accepted_by`, with Resend's message id, which the owner can look up in the Resend dashboard. No receipt, bounce or display is observed.
- **Transient failures retry.** A network error before anything was sent (for push, only a connection that was never made: refused, a failed DNS lookup or a connect timeout), an SMTP 4xx reply, or HTTP 429 or 5xx:

the row retries after 1 and then 5 minutes (the configured backoff), or later if the provider's `Retry-After` asks for more (capped at 6 hours). After 3 attempts it is dead-lettered as "retries exhausted".

- **Permanent failures stop at once.** A rejected login, an SMTP 5xx reply, Resend 400/401/403/422, a disabled or unconfigured channel, no push subscription, or no address for the account: one attempt, then a dead letter whose reason names what to check. For example, a Gmail 535 reads "check SMTP_USER and SMTP_PASSWORD (Gmail needs an app password)".
- **At most once.** When the outcome is unknown, the row is dead-lettered as "unknown outcome" and never re-sent: the SMTP connection dropped inside the send; a push failed in any way other than an HTTP status or a connection that was never made (a read timeout, or the connection dropping after the request was sent, when the push service may already have stored it); or the process stopped while the row was `sending`. SMTP and web push cannot deduplicate a repeat. So if one of an account's phones gives an unknown outcome, the row stops for all of them, and a phone whose push service answered 5xx loses that notification, visibly, as a dead letter. Email over Resend carries a per-row `Idempotency-Key`, which Resend keeps for 24 hours, so its timed-out requests do retry without a second email. A `sending` row counts as abandoned after 60 minutes; a younger one may belong to the old container during a deploy.
- **Recipient isolation.** `resolve_recipient()` returns the owning account's address from `users.db`. `DELIVERY_EMAIL_TO` is used only on the single-user path: no user id, or the default portfolio id. For any other account, a failed lookup retries and a missing address dead-letters; neither reaches the owner. The inline path (Atlas off, or the outbox unreachable) follows the same rule.
- **Redaction.** Reasons and log lines mask email addresses, URL paths (a push endpoint is a bearer capability) and the configured secrets.
- **Push TTL.** Pushes carry a 12-hour TTL (`delivery.push_ttl_seconds`). pywebpush's default of 0 lets a push service drop a notification for a phone that is offline when it arrives. This comes from reading the library; its effect on what users saw was not measured.
- **Visibility.** `GET /delivery/outbox` (owner session or machine key) returns counts per channel and status, with `delivered` shown as `accepted`; the newest dead letters and retrying rows with their reasons; and the last acceptance per channel. It states that user receipt is not observable, and it carries no payload, address or user id: a dead letter for an account with no address reads "no email on file for this account", and its row id identifies it.
- **History.** Nightly retention deletes accepted rows after 30 days. Dead letters stay for 180 days; after 30 days only their payload is cleared.

The monthly Learning Evidence email, the watchdog heartbeat and the backup email are sent directly, not through the outbox, so they get no retry and no dead letter; a failure is logged with its reason. The backup email is not the off-site copy (below). **Planned:** a witness outside the app for missed jobs (SA-042).

Dated production observation. The September audit recorded email failures. The cause was identified on 2026-09-21: Railway disables outbound SMTP on its Hobby plan, and production had logged `[Errno 101] Network is unreachable` on every send since 2026-07-16. After a plan upgrade and redeploy, a triggered brief reached the inbox that day. A read-only probe on 2026-09-23 counted, since that redeploy, 8 email rows accepted and 0 dead, and 11 push rows accepted; before 21 Sep, 77 email rows were dead and 77 push rows accepted. Those counts are provider acceptance, not a measured receipt rate.

Remaining limits:

- Resend's default shared sender (`RESEND_FROM`) delivers only to the Resend account owner's address. Per-account email over Resend needs a verified-domain sender, which is an owner configuration step, not code.

- When several push subscriptions exist, acceptance by one counts as accepted; the others are not retried, so they cannot receive a duplicate.
- Resend deduplicates a key for 24 hours. Retries end well inside that window (two waits, each capped at 6 hours); a policy with longer waits would lose that protection.

Backups and recovery

The Railway volume is the only home of the trade ledgers, the prediction tree and the SQLite stores, so a backup must leave the volume and must restore. Three SQLite stores (`users.db`, `atlas.db` and `chat_sessions.db`) run in WAL mode and cannot be copied as raw files: restored alone, a raw copy of a fresh WAL database has no tables at all. On 2026-09-23 the older job built an 8.98 MB zip, its email failed, and the only copy stayed on the volume.

Each night at 23:30 the backup job runs these steps:

1. **Build.** Every non-cache file under `data/` goes in. A file is treated as SQLite by its header, not its name, and is copied through the backup API; `-wal`, `-shm` and `-journal` files are never copied raw. Credential-shaped files (`.env`, `*.pem`, `*.key`, `*secret*` and similar) are left out and logged. Other files are read until one read sees no size or modification change. `MANIFEST.json` inside the zip records each file's size and SHA-256. For each database it records `integrity_check`, a schema digest and every table's row count, and for each portfolio ledger its row count. A sidecar `<archive>.manifest.json` records the zip's own size and SHA-256. The zip is built under a `.part` name, so an interrupted run leaves no archive.
2. **Drill.** restore restores the archive into a fresh temporary directory and recomputes everything the manifest claims. Errors mean the copy is not faithful: a missing manifest, a checksum mismatch, a corrupt zip, an unexpected or unsafe member, a failed `integrity_check`, or a schema or row-count difference. Warnings mean the copy is faithful but the data is suspect. The drill also checks ledger continuity. For example, Monday's archive holds `transactions.jsonl` at 1,200 bytes, so Tuesday's must start with those same 1,200 bytes, because the four portfolio ledgers are append-only. If a script rewrote row 3 overnight, Tuesday's drill warns "was rewritten"; a ledger that shrank or disappeared warns the same way. Older off-site copies still hold the earlier history.
3. **Rotate** the local copies to the newest 7, each with its sidecar.
4. **Off-site.** Only an archive that passed its drill leaves the volume. offsite encrypts it with AES-256-GCM under `BACKUP_ENCRYPTION_KEY` and uploads it to `BACKUP_OFFSITE_TARGET`. That is `s3` (any S3-compatible bucket, signed with AWS Signature V4 using only the standard library and `requests`) or `dir` (a separately mounted directory, refused when it overlaps `data/`). The ciphertext goes first and the manifest last; each must read back at its uploaded size. Only then is the copy "confirmed". An upload interrupted halfway is never confirmed, and its leftover ciphertext is deleted once a newer copy is confirmed. Retention keeps the newest `BACKUP_OFFSITE_KEEP` confirmed copies (30 by default; 0 or less keeps all). It never touches objects not named like a backup, and it deletes a copy's manifest before its ciphertext.
5. **Email** the plaintext zip when it is under 20 MB. This is a direct send; it does not count as off-site.
6. **Record** `data/backups/backup_status.json`. A failed drill then raises, so the scheduler's job-error alert fires.

Visibility. The watchdog invariant `backup_recoverable` reads that status every morning. It warns when the job has not run for 36 hours, when the drill failed, when last night's copy was not confirmed (the reason is scrubbed of secrets), and when the drill flagged a ledger. While no target is configured it says "No off-site copy has ever been confirmed", **once a week**: the check asks for that pace itself (`repeat_days=7` on its result). Everything else still warns daily: a failed drill, a stopped job, a flagged ledger even with no target, and a half-done setup such as a target without a key. For example, a reminder on Thursday is followed by silence until the next Thursday. If Friday's drill fails, Saturday's watchdog warns anyway.

A ledger rewrite is reported on one morning only, because the next night compares against the rewritten archive. The earlier history then survives only in the older off-site copies, for `BACKUP_OFFSITE_KEEP` nights.

Owner configuration (not provisioned by code). Create a private bucket and an access key limited to it. The archive is about 9 MB, and several S3-compatible providers have a free tier at that size. Set these Railway variables:

- `BACKUP_OFFSITE_TARGET=s3`;
- `BACKUP_S3_ENDPOINT` (a bare `https://` URL), `BACKUP_S3_BUCKET`, `BACKUP_S3_REGION` (`auto` by default) and optionally `BACKUP_S3_PREFIX` (`stockagent/`);
- `BACKUP_S3_ACCESS_KEY_ID` and `BACKUP_S3_SECRET_ACCESS_KEY`;
- `BACKUP_ENCRYPTION_KEY`, from `python -c "import os,base64;print(base64.b64encode(os.urandom(32)).decode())"`.

Keep a second copy of the key outside Railway, for example in a password manager: without it no off-site copy can ever be read. Setting a Railway variable redeploys the service.

Restore runbook. On a machine with the same variables set, run `python -m services.data.restore fetch --dest <empty directory>` from the repository root. It downloads the newest confirmed copy (or a named one), checks the ciphertext against its manifest and decrypts it; a wrong key or one flipped byte stops it. It then drills the result into `<dir>/restored`. Exit codes: 0 is clean, 2 is faithful with warnings, 1 is failed. `python -m services.data.restore drill <archive>` checks a local archive the same way. Neither ever writes into `data/`: a destination that overlaps it, or is not empty, is refused. Putting restored files back into a stopped deployment is a deliberate human step.

Limits.

- Each file is a point-in-time copy, but the archive is not one snapshot across files. The job runs at 23:30, when no trading job writes.
- A night on which the job never runs, for example because the container restarted at 23:30, is not reported: at 06:30 the newest status is 31 hours old, under the 36-hour limit, so the check still reads the previous night's copy as confirmed. Two missed nights in a row are reported. **Planned:** SA-034.
- One malformed SQLite file stops the whole archive. The job raises, so the job-error alert fires each night. No new archive, off-site copy or status is written until the database is repaired, so the ledgers are not backed up in the meantime; earlier archives and off-site copies are kept. **Planned:** SA-036.
- The email copy is an unencrypted direct send.
- Credentials live in Railway variables and are not backed up.
- A deleted account's data survives in off-site copies until retention removes them (30 nightly copies by default).
- Off-site recovery counts as verified only after a recorded `fetch` drill against the real bucket. None has been run, because no bucket exists yet.

Interface and chat rendering

The frontend uses React JSX, runtime browser transformation and PWA assets. Chat uses a streaming tool loop with potentially paid provider calls. Prompt editing can feed scheduled publishing. These are not inert testing screens. Bearer sessions, roles and machine-key authentication are implemented; portfolio identity comes from the authenticated user. Chat's tools follow the same rule: the brief tool reads only the caller's portfolio and refuses a request without a valid user id, rather than fall back to the default user (CHAT_ARCHITECTURE section 2).

Some intelligence read routes remain public. The RL client has demo/fallback paths on API failure, so populated charts alone do not prove live data. **Planned:** honest unavailable states (SA-024) and a public-read and demo-data policy (SA-032).

Chat replies are untrusted text: the model quotes news snippets and tool output, and a quoted `<img src=x onerror=…>` passed to the browser as HTML would run its handler next to the stored bearer token (audit F01). `renderMd` in sphere.jsx therefore delegates to a plain script, chat-markdown.js. index.html loads it after marked 12.0.2 and DOMPurify 3.4.16, both exact versions with SRI hashes. There are two layers:

- marked shows any raw HTML in a reply as visible text. The chat prompt's format rules describe Markdown, but nothing in it forbids HTML, so this boundary is the control.
- DOMPurify then keeps only Markdown's own tags. Links must be http(s) and open in a new tab with `noopener noreferrer`. Images, styles, classes and event attributes are dropped.

For example, a reply quoting `<img src=x onerror=…>` shows that string as text; bold, lists, code fences, tables and https links still render. If either library fails to load, the reply is shown as escaped text. The raw parser is taken off `window`, so an older `sphere.jsx` still cached by the service worker also falls back to text; the worker's cache version is v8. That bubble is the client's only HTML sink, and a unit test keeps it so. `npm run test:frontend` drives the real ChatOverlay in Chromium with hostile fixtures and no network.

Observability and the watchdog

Durable logs connect run IDs, tickers, warnings, LLM usage and health. Bundle call counters are not complete nested-provider/fallback cost accounting.

The watchdog reads milestones.yaml, runs checks, persists state and may perform configured preparation. A sent milestone notice is not milestone completion. A `manual_confirmation` milestone has no signal of its own: it stays pending until its entry is deleted, and after its deadline it re-sends a critical "LAPSED" notice every 7 days. Each lapsed milestone is judged, then deleted with its evidence or decision recorded, or re-dated with a dated reason at the head of its action text. A registry edit reaches production only on deploy.

Local backup files and healthy HTTP responses do not prove recovery or successful jobs. The nightly drill proves that an archive restores; only a recorded off-site `fetch` proves recovery without the volume. **Planned:** the observability design adds job-run and source-health ledgers, post-job checks, provider-credit alerts, a read-only status fetcher and an outside witness (section 11).

11. Planned redesign

Nothing in this section is implemented. Story cards define scope and acceptance, and STATE.json holds status. Accepting code needs tests and a fresh review. Claiming a benefit needs matured production observations as well.

One engine with sector lenses

All five graphs already run one bundle and one reasoning-model call; the LangGraph pool is only a fallback. They differ mainly in five scoring schemes (37 dimension slots under 24 names), and the LLM re-scores numbers the code has already computed. Generic-graph stocks are valued against automobile peers, and missing data is scored 0.5. The one-engine design, adopted on 2026-09-26, replaces the per-sector graphs in four parts:

- sector knowledge becomes a lens YAML (peers, benchmark, KPIs, news terms and macro drivers), resolved from NSE's industry field;
- code computes five factors (Value, Quality, Growth, Momentum and Risk) as percentiles against real peers and the stock's own history;
- one LLM reader returns dated, sourced events, and code turns them into a sixth factor, Catalyst;
- code combines the six at equal weights and maps the result to the existing bands. With fewer than 4 of 6 factors it gives no directional verdict. The LLM explains the verdict but cannot change it.

SA-044 fixes the peers first. SA-026, SA-045 and SA-046 run in shadow. SA-047 switches only when the engine is right on at least 50% of disagreements with today's analyst, and SA-027 then deletes the sector packages. Each lens also names its sector's benchmark, which is where automobile and renewable energy get an index with data.

Learning on a corrected target

SA-012 specifies and tests the learning target before anything changes it (the grading defect in section 5). SA-013 persists immutable, prospective forecast identities, and SA-014 grades frozen decisions with the information available at issue. SA-015 makes adaptive updates idempotent and keeps final weight bounds. SA-016 replaces blame-based shared credit with identifiable outcomes. SA-017 attaches lineage and quarantines incomparable feedback. SA-018 to SA-021 build complete matched cohorts, effective samples from session overlap, and calibration and weight-recovery checks.

Returning to `adapt`. SA-043 runs a shadow learner while production stays in `observe`. It learns six factor weights pooled across all stocks, starting from the defaults and learning on the corrected target. For each issued decision it records the default verdict and the shadow verdict from the same factor scores. SA-022 then scores only the decisions where the two disagree. Learning may return only when all of these hold:

- at least 100 effective disagreements, over at least 3 months and 3 lenses;
- the shadow is right on at least 60% of them;
- its wins are consistent month to month and robust to dropping its best lens;
- no factor's weight has collapsed to its bound, or below half its default, for 4 weeks.

The pass must hold at two consecutive monthly looks, and the owner still decides. `adapt` would then resume from the shadow's weights, never from the stored pre-fix weights. A reverse tally in the first month (below 45%) returns learning to `observe`. SA-023 puts learned lessons on measurable probation.

Portfolio economics

- SA-048 charges realistic trading costs and tax in paper P&L.
- SA-050 applies one sizing rule to every autopilot buy, including the SWITCH buy, which today spends a whole sale's proceeds.
- SA-049 puts idle cash into shelf ideas that pass the checks in a normal market, and holds cash on purpose only in a crisis, behind a switch that ships off.
- SA-051 shows the portfolio against the Nifty, with honest labels for "Invested" and returns.

Operations and observability

The observability design makes failures visible without reading raw logs:

- SA-036 records a durable outcome for every critical job, and SA-037 keeps deploys from dropping scheduled jobs;
- SA-040 records per-source fetch health and detects new error types;
- SA-034 detects silent degradation in the watchdog, including a missed backup night and the review outcome record;
- SA-035 surfaces LLM and search provider credit exhaustion;
- SA-041 provides a read-only production status fetcher, and SA-042 adds an outside witness for missed jobs (Healthchecks.io → Telegram);
- SA-011 turns durable errors into a deterministic operations backlog;
- SA-029 verifies readiness and recovers background ownership;
- SA-025 counts actual provider calls and fallback cost;
- SA-033 locks reproducible runtime dependencies.

Interface and data policy

SA-024 shows verifiable learning evidence and honest unavailable states. The stretch stories SA-028 (packaging and a compiled browser client), SA-030 (stale Atlas projections) and SA-032 (the public-read and demo-data policy) need explicit promotion. SA-031 is the final cross-document consistency check.

Owner decisions and setup

These need the owner, not code:

- whether to enforce the decision gate, judged on rows issued since the bank and resolution fixes (section 4);
- an off-site backup bucket and key, then a recorded `fetch` drill (section 10);
- a verified-domain sender, if per-account email goes over Resend;
- whether learning returns to `adapt`, under the conditions above.

Historical specifications retain what was intended at the time. The September audit supersedes unsupported August claims about correct learning, automatic security succession or unused fallback cost. Legacy code should only be retired with measured replacement coverage.

12. PI-2026-09 roadmap

Each story card defines its scope, acceptance and dependencies, and STATE.json holds its status. Evidence receipts under evidence/ record the tests, the reviewed revision and the production checks. The table maps each story to the sections of this document it changes. An implemented story's behaviour is described in those sections as current design. Its production verification is tracked separately (section 1).

Five fixes outside the SA numbering are also part of the current design: listing-day closes (FIX-001, sections 5 and 7), ticker resolution (FIX-002, section 4), bank income statements (FIX-003, section 4), dossier dates (FIX-004, section 5) and the chat brief's user scope (FIX-005, section 10). DOC-001 produced this document's current-code edition.

Story	Change	Status	KT sections	Dependencies
SA-001	Sanitize every chat Markdown rendering boundary	Done	10 Interface	None
SA-002	Make data-health records describe usable evidence	Done	4 Research	None
SA-003	Gate recommendations and learning on essential data	Done	4 Research; 5 Learning; 6 Portfolio	SA-002
SA-004	Count daily-review outcomes truthfully	Done	9 Jobs	None
SA-005	Establish a clean, isolated test and CI baseline	Done	13 Validation	None
SA-006	Repair delivery transport and expose dead letters	Done	10 Delivery	None
SA-007	Make backups independently recoverable	Done	3 Storage; 10 Recovery	None
SA-008	Quarantine unresolved securities with explicit lifecycle records	Done	3 Configuration; 4 Identity; 5 Learning; 6 Portfolio; 8 Discovery	SA-003
SA-009	Inventory and reconcile prediction-store ownership	Done	3 Storage	None
SA-010	Pass sector benchmarks through technical-data calls	Done	4 Research	None
SA-011	Turn durable errors into a deterministic operations backlog	Open	9 Jobs; 10 Operations	SA-002, SA-004
SA-012	Specify and test the learning target before changing it	Open	5 Learning; 7 Marksheets	SA-005
SA-013	Persist immutable, genuinely prospective forecast identities	Open	3 Storage; 5 Learning	SA-012
SA-014	Grade frozen decisions and issue controls before their outcomes	Open	5 Learning; 7 Marksheets	SA-003, SA-013
SA-015	Make adaptive updates idempotent and preserve final weight bounds	Open	5 Learning	SA-014
SA-016	Replace blame-based shared credit with identifiable outcomes	Open	5 Learning	SA-015
SA-017	Attach historical lineage and quarantine incomparable feedback	Open	3 Storage; 5 Learning	SA-009, SA-013
SA-018	Build complete, matched monthly evaluation cohorts	Open	7 Marksheets	SA-014, SA-017
SA-019	Evaluate suggestions across weekly and fortnightly cohorts	Open	7 Marksheets	SA-018

Story	Change	Status	KT sections	Dependencies
SA-020	Compute effective samples from trading-session overlap	Open	5 Learning; 7 Marksheets	SA-013
SA-021	Validate calibration and weight recovery with minimum evidence	Open	5 Learning; 7 Marksheets	SA-016, SA-018, SA-020
SA-022	Run prospective adapted and frozen-policy experiments	Open	5 Learning; 7 Marksheets	SA-016, SA-018, SA-020, SA-043, SA-047
SA-023	Put learned lessons on measurable probation	Open	5 Learning	SA-022
SA-024	Show verifiable learning evidence and honest unavailable states	Open	7 Marksheets; 10 Interface	SA-019, SA-020, SA-022
SA-025	Count actual provider calls and fallback cost	Open	4 Research; 10 Cost	SA-011
SA-026	Consolidate sector definitions into sector lenses	Open	2 Repository; 4 Research	SA-003, SA-009, SA-010
SA-027	Retire redundant fallback only when measured replacements work	Open	4 Research; 11 Planned redesign	SA-025, SA-026, SA-047
SA-028 (stretch)	Repair packaging and compile the browser client	Open	3 Runtime; 10 Interface	SA-001, SA-005
SA-029	Verify readiness and recover background ownership	Open	3 Runtime; 9 Jobs	SA-004, SA-007
SA-030 (stretch)	Reconcile or retire stale Atlas projections	Open	3 Storage	SA-009, SA-013
SA-031	Verify final architecture and operator-documentation consistency	Open	All: final consistency check	SA-024, SA-027, SA-029
SA-032 (stretch)	Define and enforce the public-read and demo-data policy	Open	10 Interface	SA-001, SA-024
SA-033	Lock reproducible runtime dependencies	Open	3 Runtime; 13 Validation	None
SA-034	Detect silent degradation in the watchdog	Open	9 Jobs; 10 Operations	SA-036, SA-040
SA-035	Surface LLM and search provider credit exhaustion	Open	10 Operations; 10 Cost	None
SA-036	Record durable outcomes for every critical job	Open	9 Jobs	None
SA-037	Keep deploys from dropping scheduled jobs	Open	3 Runtime; 9 Jobs	SA-036
SA-038	Judge and retire lapsed production-verification milestones	Done	9 Jobs; 10 Operations	None

Story	Change	Status	KT sections	Dependencies
SA-039	Contain adaptive learning: observe-only, live weights back to defaults	Done	5 Learning	None
SA-040	Record per-source fetch health and detect new error types	Open	9 Jobs; 10 Operations	None
SA-041	Provide a read-only production status fetcher	Open	10 Operations; 13 Validation	SA-036, SA-040
SA-042	Add an outside witness for missed jobs (Healthchecks.io → Telegram)	Open	9 Jobs; 10 Operations	SA-036
SA-043	Run a shadow learner and record paired decisions in observe mode	Open	5 Learning	SA-015, SA-039, SA-045
SA-044	Compare valuations only with same-industry peers	Open	4 Research	None
SA-045	Compute the five universal factors deterministically	Open	4 Research	SA-026
SA-046	Read text into dated events and a Catalyst factor	Open	4 Research	SA-026
SA-047	Decide with the factor engine; switch when it is not worse	Open	4 Research; 5 Learning	SA-045, SA-046, SA-014, SA-020
SA-048	Charge realistic trading costs and tax in paper P&L	Open	6 Portfolio; 7 Marksheets	None
SA-049	Put idle cash to work in normal markets	Open	6 Portfolio; 8 Discovery	SA-050, SA-051
SA-050	Apply one sizing rule to every autopilot buy	Open	6 Portfolio	None
SA-051	Show the portfolio against the Nifty, with honest labels	Open	7 Marksheets; 10 Interface	None

Maintain documentation with each story

Each implementation updates the affected sections so that they describe the new behaviour as current design. It does not add a change note. It also sets the story's status in the table above, adjusts the relevant human test cases and regenerates the PDF when this Markdown changes. Commits, reviews and deploys belong in STATE.json and the evidence receipts, not in this document. Measured production state belongs in section 1's table. SA-031 is the final cross-document consistency check, not a reason to postpone normal documentation.

13. Validation and KT sequence

The DOC-001 receipt and review record how this document's first current-code edition was checked; each story's receipt records its own tests. Local validation uses Python 3.13 on Windows; production Docker and CI use Python 3.11 on Linux. Focused checks do not imply full-suite or platform-parity acceptance.

The test suite's hermetic boundary

Every `pytest` run passes through tests/hermetic.py, which tests/conftest.py imports before any application code. The same four rules hold on a laptop with a `.env` and on a fresh CI checkout:

Rule	What it does	Example
No `.env`, no shell settings	dotenv is off, and every variable the application reads is removed, so settings come from `config.yaml` and the code fallbacks.	A developer's `.env` says `RL_LEARNING_MODE=observe`; the suite still runs the shipped `adapt`.
No outbound network	Non-loopback sockets, DNS and `curl_cffi` (yfinance's transport, which bypasses Python sockets) raise `OutboundNetworkBlocked`. A test that triggers one fails, even if the application swallowed the error.	An unmocked thesis review would call OpenRouter; the test fails, naming `thesis_reviewer.py` and `openrouter.ai:443`.
An empty working directory	The application keeps runtime state under `data/` and `outputs/` relative to the working directory, as under `/app` on Railway. Each test runs in its own empty temporary directory, seeded only with the three tracked files the application reads by relative path.	A test that toggles a managed ticker changes its sandbox copy, not the checkout's `data/managed_tickers.json`.
The checkout is off limits	An audit hook refuses any access to the checkout's `data/`, `logs/` and `outputs/`, and any write elsewhere in it.	A test reading local MARUTI predictions fails; the discovery test builds its own predictions tree.

Why it is this strict: on 2026-09-27 a socket guard alone saw 74 blocked attempts, while yfinance's `curl_cffi` requests were invisible to it. The full boundary then found 286 Yahoo requests, 178 OpenRouter attempts and 124 NSE session attempts. The tests had also been writing 149 paths in the checkout (35 targets once temp files are grouped) and reading 24 untracked runtime files. Most network traffic came from tests that booted the full app, whose startup ran the RL self-heal thread; those tests boot the app as an API-only worker.

The boundary does not reach child processes (the eval CLI smoke test and the browser suite); they inherit only the cleaned environment. It sees Python's own socket, DNS and file calls, and `curl_cffi`. It does not refuse a directory listing or existence check, a native library's own file access (pyarrow, or SQLite given a `file:` URI), a Windows short-name spelling of the checkout, or a numeric-IP connection on the Windows asyncio loop. The application builds one absolute checkout path, in the startup calendar check, and the test fixture stubs it. A Windows-only retry in core/utils/atomic_io.py absorbs the transient `PermissionError` of a rename on Windows; Linux renames never retry.

Continuous integration

The CI workflow, .github/workflows/ci.yml, runs three jobs on Linux with Python 3.11 and no secrets:

- the whole `tests/` tree;
- the broad-exception guard (scripts/ci/check_broad_except.py) and `check_kt_docs.py`;

- the chat-rendering browser suite in Chromium.

The guard fails when new code catches every exception without logging, re-raising or using it. The 154 handlers that already did so are grandfathered in a burn-down list; they were not reviewed one by one. The list counts handlers per function, so a change that fixes one grandfathered handler and adds another in the same function passes. Test-only tools are pinned in requirements-test.txt. The runtime packages still come from the unpinned `requirements.txt`. **Planned:** a lock (SA-033).

KT documentation tools

Maintainer commands, with the required local dependencies installed:

```
python scripts/docs/run_kt_checks.py
python scripts/docs/build_kt_pdf.py
python scripts/docs/check_kt_docs.py
```

The test runner copies tracked source to ignored `analysis_data/`, excludes `.env`/runtime stores, disables dotenv, and blocks external transports and test subprocesses. It exercises a fixed selection of existing tests, not all independent financial invariants. The PDF builder needs Python `markdown2` and local Playwright/Chromium; it does not start the application. The checker validates links, the roadmap rows against STATE.json, the job table against the scheduler, selected repository config values, the header revision and the PDF's freshness.

KT sessions

KT session	Walkthrough
Product/data	Sections 1–4: production state, a symbol, its identity, source dates and statuses, dimensions, the decision gate and the research verdict.
Predictions/learning	Section 5: envelope row to review checks, feedback, weights or proposals, lessons and dossier dates; explain the grading gaps and the learning mode.
Portfolio/marksheets	Sections 6–7: advice to transaction, the data and identity holds, the P/L example and different evaluation rules.
IPO/discovery	Section 8: calendar to captured facts, history, post-listing candidates, and the dark P3 path from deep dive to a stored, narrated and graded verdict.
Operations/roadmap	Sections 9–12: scheduled work to persisted output and per-ticker outcomes, outbox states and the dead-letter view, the backup drill and restore runbook, the watchdog, and the planned redesign.

Practical, non-code-intensive duties are in the separate Team Human Testing Guide. Testers should be able to explain outputs and compare evidence without reading prompts or reviewing code.